

Hacia un traductor especializado en jergas y dialectos del español: generación de datos sintéticos multi-fuente y fusión de modelos de lenguaje pequeños

Anderson García Mariana Malagón
Paula Lozano

Ingeniería de Sistemas, Escuela Colombiana de Ingeniería Julio Garavito

17 de septiembre de 2026

Resumen

Los sistemas de traducción automática contemporáneos alcanzan un desempeño alto en el registro estándar de un idioma, pero tienden a fallar cuando el texto de entrada contiene jerga o expresiones dialectales, produciendo traducciones literales que distorsionan o invierten el significado original. Esta limitación no es meramente incómoda: existen casos documentados en contextos de salud y de justicia donde un error de esta naturaleza tuvo consecuencias severas para las personas afectadas. Este documento presenta la propuesta inicial de un proyecto que investiga la viabilidad de construir un traductor especializado en jerga y dialectos del español y el inglés mediante el ajuste fino (*fine-tuning*) de un modelo de lenguaje pequeño (*Small Language Model*, SLM) sobre datos sintéticos generados por múltiples modelos de lenguaje de gran escala (LLMs). Se propone, además, investigar si la fusión de pesos (*model merging*) de modelos entrenados con datos provenientes de distintos generadores sintéticos permite consolidar el conocimiento de varias fuentes en un único modelo eficiente, sin incurrir en los costos de un *ensemble*. Estas son, ante todo, preguntas de investigación; que el resultado además sirva como propuesta de producto frente a alternativas como Google Translate o DeepL es una pregunta distinta, que este documento aborda por separado. Esta primera entrega delimita el problema, revisa el estado del arte en traducción dialectal, generación de datos sintéticos y fusión de modelos, identifica los vacíos que motivan el proyecto, y describe la propuesta general de solución. La arquitectura detallada, el prototipo y los resultados experimentales se desarrollarán en las siguientes fases del proyecto.

Palabras clave: traducción automática, dialectos, jerga, modelos de lenguaje pequeños, datos sintéticos, fusión de modelos, destilación de conocimiento.

1. Introducción

1.1. Contexto

Hoy en día entrenar modelos de lenguaje ya no depende de un solo generador de datos: es común mezclar salidas de varios LLMs distintos y luego fusionar los modelos resultantes, en lugar de apostarle a uno solo. Esa combinación (síntesis de varias fuentes más fusión) es relativamente nueva y todavía no se ha probado a fondo para tareas específicas del español, como traducir jerga o dialectos regionales.

Y ahí surge una pregunta que cualquier hispanohablante se ha hecho sin necesidad de ser investigador: si hoy entrenamos IA con técnicas tan sofisticadas para innumerables propósitos, ¿por qué seguimos sin lograr que entienda que "dar papaya." o "qué galleta" significan cosas completamente distintas según de dónde venga quien las dice?

Este no es un problema nuevo ni menor. En 1980, un joven cubanoamericano de Florida, Willie Ramírez, quedó cuadripléjico porque el personal médico tradujo 'intoxicado' como *intoxicated* en vez de *poisoned*, y lo trataron por sobredosis cuando en realidad tenía una hemorragia cerebral. El hospital terminó pagando una indemnización de 71 millones de dólares. Han transcurrido más de cuarenta años y, aunque los modelos actuales traducen el español estándar con un nivel de precisión considerablemente

alto, un estudio de ACL 2025 (Mi, Villavicencio y Moosavi) muestra que los LLMs más recientes siguen fallando específicamente al interpretar el contexto de expresiones idiomáticas. Es decir, el problema no radica en que la IA traduzca de forma deficiente, sino en que, pese a un desempeño notable en términos generales, persiste sin resolverse este aspecto particular.

Por eso, este trabajo no busca solo construir una herramienta más de traducción, sino generar evidencia sobre si combinar generadores sintéticos con fusión de modelos realmente funciona para esta tarea concreta, algo que hasta ahora nadie ha probado de forma sistemática para el español.

1.2. Nicho identificado

Antes de entrar en el estado del arte completo, vale la pena situar con precisión el vacío que motiva este proyecto. Lo ubicamos en la intersección de tres líneas de trabajo que, hasta donde alcanzamos a revisar (ACL Anthology, Semantic Scholar, arXiv, Google Scholar, y también literatura gris como blogs técnicos y reportes de industria), no hemos encontrado combinadas en ningún lado:

- (I) **Traducción (no solo reconocimiento) de jerga y dialectos del español mediante *fine-tuning*.** Para otros idiomas ya hay resultados concretos. Yakhni y Chehab ajustaron el modelo Aya23-8B para traducir dialecto libanés, generando parte de sus datos de entrenamiento a partir de un libro de gramática [8]. Jerpelea et al. mostraron que un corpus paralelo de cerca de 79 mil ejemplos ya es suficiente para ajustar modelos preentrenados al aromanian [9], y algo parecido se hizo para el šariš, un dialecto eslovaco de bajo recurso [10]. Incluso hay un banco de pruebas de traducción de jerga inglés-chino, con más de 40 mil pares, donde el modelo especializado supera a las líneas base sin ajuste fino [11]. Para el español no encontramos nada parecido: lo que hay se queda en *reconocer* o clasificar la variante dialectal [12, 13], pero no llega a traducirla.
- (II) **Comparación controlada de múltiples LLMs como generadores de datos sintéticos, aplicada a una tarea de traducción.** La elección del generador sintético sí se ha tratado como variable experimental en otros contextos: para clasificación de falacias lógicas, con diferencias de hasta 14 puntos porcentuales de F1 según el LLM usado [16], y para diálogo médico en árabe, donde GPT-4o superó consistentemente a Gemini 2.5 Pro en calidad y tasa de alucinación [17]. También hay evidencia de que la diversidad de fuentes sintéticas por sí sola afecta el comportamiento del modelo resultante [18]. Esa comparación aplicada a una tarea de traducción de dialecto o jerga, sin embargo, no la encontramos en ninguna parte.
- (III) **Fusión de modelos entrenados sobre datos sintéticos de distintas fuentes, en una tarea generativa de traducción.** La fusión de pesos (*model merging*) ya es una técnica bastante madura para combinar modelos especializados en distintas tareas o idiomas [19–23], y también sirve para combinar modelos entrenados sobre distintos *datasets* en clasificación [25]. En traducción multilingüe la evidencia es más de advertencia que de garantía: la fusión resulta frágil cuando los modelos difieren en el idioma de destino [24], y el trabajo más reciente sobre fusión guiada por destilación multi-maestro reconoce abiertamente que llevarla a tareas generativas queda pendiente [26].

En resumen, cada una de estas tres líneas está resuelta por su lado y con buena evidencia detrás. Lo que nosotros proponemos es juntarlas para el español: traducir (no solo reconocer) la jerga y el dialecto, comparar de forma controlada los generadores que alimentan esa traducción, y consolidar el resultado fusionando modelos.

1.3. Contribuciones esperadas

Este proyecto espera aportar:

- (1) evidencia empírica sobre la viabilidad de un SLM especializado en traducir jerga y dialectos del español y el inglés;
- (2) una comparación controlada del efecto que tiene el generador de datos sintéticos (entre varios LLMs) sobre la calidad de la traducción resultante;

(3) evidencia sobre si la fusión de modelos (simple o guiada por destilación) permite consolidar el conocimiento de varias fuentes de datos sintéticos en un único modelo eficiente; y

(4) un *pipeline* de generación, validación y evaluación de datos reutilizable para otras variantes dialectales no cubiertas en este trabajo. La Sección 5 detalla estas contribuciones.

1.4. Organización del documento

El resto de este documento se organiza así: la Sección 2 caracteriza el problema y formula las preguntas de investigación. La Sección 3 presenta el estado del arte en traducción dialectal, generación de datos sintéticos y fusión de modelos. La Sección 4 describe la propuesta general de solución. La Sección 5 enumera las contribuciones esperadas. La Sección 6 describe el trabajo planeado para las siguientes fases del proyecto, y la Sección 7 concluye el documento.

2. Caracterización del problema

2.1. ¿Cuál es el problema?

Los traductores automáticos y los LLMs de uso general están hechos, sobre todo, para el registro estándar del español y el inglés. Cuando alguien usa una expresión de su dialecto (caribeño, andino, rioplatense, mexicano, centroamericano, peninsular) o jerga particular, el sistema tiende a traducirla literal o a leerla desde el registro que mejor conoce, y termina perdiendo lo que la persona quiso decir. Esto no es una sospecha sin evidencia: un estudio de 2026 evaluó a los LLMs con más de 900 palabras que cambian según el país (por ejemplo, “carro” en gran parte de Latinoamérica frente a “coche” en España) en 21 países hispanohablantes, y encontró que los modelos reconocen bien las variantes de España, México/Centroamérica y el Río de la Plata, pero fallan de forma sistemática con el español chileno, una diferencia que no se explica por la cantidad de datos disponibles en internet para cada país [14]. Es decir, el sesgo hacia ciertos dialectos no es solo un problema de escasez de datos, sino de cómo el modelo termina representando (o ignorando) las variantes menos dominantes. Fallas como esta fueron parte de lo que llevó a más de 30 instituciones latinoamericanas a construir Latam-GPT en 2026, un modelo entrenado desde cero con datos locales para no depender de un español que en los modelos de EE.UU. viene sobre todo de España o de textos traducidos del inglés [15].

Que exista un esfuerzo de esa magnitud confirma que el problema es real, pero también deja ver algo clave: esa solución pasa por entrenar un modelo fundacional completo, algo que exige presupuesto y cómputo que casi ningún equipo fuera de una coalición de universidades y gobiernos tiene. Este proyecto va por otro lado: en lugar de un modelo nuevo desde cero, ajustar uno pequeño ya existente con datos sintéticos y fusión de modelos, específicamente para traducir jerga y dialectos del español. Es una escala mucho más modesta, pero más accesible y rápida de probar, y ataca el problema desde un ángulo que Latam-GPT no cubre: no reemplazar el modelo base, sino especializarlo para una tarea puntual sin el costo de entrenar uno nuevo.

2.2. Por qué las soluciones actuales no cierran esta brecha

Vale la pena ser explícitos sobre algo que cualquier lector va a preguntar de inmediato: ¿no resolvió esto ya Google, con la actualización de Translate basada en Gemini de diciembre de 2025? La respuesta corta es que no, al menos no donde importa para este proyecto. Google anunció que Gemini mejora la traducción de “modismos, expresiones locales o jerga”, con “calidad de vanguardia” [27], pero el ejemplo público que usan es un modismo genérico en inglés (“*stealing my thunder*”), no variación dialectal dentro del español, y no hay evidencia pública de que se haya evaluado contra jerga regional (chilena, colombiana, rioplatense). De hecho, dos meses después de ese anuncio, el propio estudio de Kawasaki que citamos arriba [14] sigue documentando que los LLMs fallan de forma sistemática en representar la variación léxica geográfica del español, y un estudio de la misma época encuentra disparidades atribuibles a dialectos específicos en el sistema de subtítulos automáticos de YouTube [29]. Esto tiene una

consecuencia que cualquiera que haya visto un video con subtítulos traducidos automáticamente reconoce de inmediato: YouTube primero transcribe el audio y después traduce esa transcripción, así que si la transcripción ya viene mal por un acento o dialecto que el sistema no reconoce bien, la traducción hereda y agrava ese error, produciendo subtítulos que aparecen en pantalla con toda la apariencia de estar bien, pero que no dicen lo que la persona realmente dijo [30]. DeepL es todavía más directo: su propia documentación de 2026 dice, textualmente, que el español “latinoamericano” que produce “no reflejará una variante específica, sino un estilo de español general y más neutro” [28]. No es que lo intenten y les falte precisión: no lo intentan.

Tampoco faltan herramientas de nicho —hay traductores enfocados en un solo dialecto, como uno especializado en jerga dominicana—, pero son productos de entretenimiento sin metodología publicada ni evaluación reproducible, pensados para sonar “como local” en una conversación casual, no para un contexto donde el error cuesta caro. Ninguno de los dos extremos —el gigante generalista y el juguete de nicho— hace lo que describimos en la Sección 3: comparar de forma controlada distintos generadores de datos sintéticos, o fusionar los modelos resultantes.

2.3. Un problema que no se agota en el español

El sesgo hacia ciertas variantes tampoco es exclusivo del español. Colombia es un buen ejemplo de por qué el problema es más amplio de lo que hemos planteado hasta acá: además de sus variantes dialectales del español, el país tiene cerca de 65 lenguas indígenas vivas, de las cuales el Ministerio de Cultura no clasifica ninguna como completamente segura y ubica a más de la mitad en algún grado de riesgo de desaparición [31]. La escasez de datos digitales para esas lenguas es todavía más severa que para el dialecto o la jerga del español. Es un problema de otra naturaleza —no es cuestión de matiz o registro, sino de escasez casi total de datos de entrenamiento—, así que no lo incorporamos al alcance técnico de este proyecto. Pero si el *pipeline* que proponemos (semillas curadas, generación sintética, ajuste de un modelo pequeño, evaluación con hablantes nativos) demuestra ser viable para el español, es una base metodológica razonable para explorar, como trabajo futuro, lenguas indígenas colombianas con escasez de datos todavía mayor.

Y ahí es donde una decisión de diseño que ya estaba en nuestra propuesta original (Sección 4) resulta más relevante de lo que parecía al principio: elegimos un SLM y no un LLM grande en parte porque un modelo de 1 a 3 mil millones de parámetros puede correr en hardware modesto, incluso sin depender de una conexión a internet constante. En el contexto colombiano eso no es un detalle menor: aunque la conectividad rural pasó de 32.2 % en 2022 a 56.9 % en 2025 [32], eso todavía deja a más de cuatro de cada diez hogares rurales sin internet, y esas mismas zonas rurales y apartadas Vaupés, Chocó, La Guajira, Amazonas son, no por coincidencia, donde se concentra buena parte de la diversidad lingüística indígena del país. Un traductor que dependa de una API en la nube, como Google Translate o ChatGPT, no es una opción utilizable ahí. Un modelo pequeño que corra localmente sí lo es.

2.4. ¿Por qué importa y a quién afecta?

Las personas afectadas son, principalmente: hablantes de español con dominio limitado del inglés (o viceversa) que dependen de traducción automática o interpretación informal en contextos críticos de salud, justicia o servicios esenciales; profesionales que deben actuar a partir de una traducción que puede no reflejar lo que la persona quiso comunicar; e intérpretes informales (con frecuencia familiares o menores de edad) que terminan asumiendo una carga y un riesgo que no les corresponde [2]. El caso de Willie Ramírez [1] ilustra el extremo de las consecuencias posibles, pero no es el único tipo de evidencia que hay. La literatura sobre traducción de contenido generado por usuarios muestra que las expresiones dialectales y de jerga son una fuente recurrente de errores “críticos” (esos que invierten o distorsionan por completo el significado original) incluso fuera de contextos médicos: un estudio sobre traducción de tuits en español documentó que la expresión argentina “ni en pedo” (“de ninguna manera”) fue traducida de forma literal y sin sentido [6], y un estudio análogo sobre tuits en árabe relacionados con salud mental encontró que las expresiones dialectales explicaban una porción sustancial de los errores críticos

analizados [5].

2.5. Evidencia de la magnitud del problema

Según cifras del censo de Estados Unidos, en 2020 había cerca de 40.5 millones de hablantes de español en el país [3], y estudios sobre dominio limitado del inglés (*Limited English Proficiency*, LEP) estiman que una fracción sustancial de esa población (entre el 39 % y el 41 %, según la fuente y el año) reporta un dominio del inglés menor a “muy bueno” [4]. Esta magnitud poblacional, sumada a la evidencia cualitativa de los casos documentados, sustenta que el problema es real y no un supuesto sin evidencia. Sin embargo, el equipo reconoce una limitación: la frecuencia exacta de errores de traducción dialectal en interacciones cotidianas, fuera de los casos extremos reportados, no ha sido medida directamente todavía, y es justamente parte de lo que este proyecto busca aproximar de forma indirecta a través de sus experimentos de evaluación.

2.6. Por qué no es un problema trivial

Resolver este problema no equivale a conectar una interfaz de usuario a una API de traducción existente. Requiere, como mínimo, tomar decisiones no triviales en al menos cuatro frentes. De datos, porque no existen corpus paralelos amplios de jerga y dialecto del español para *fine-tuning*, así que hay que diseñar un proceso de generación sintética a partir de semillas curadas y validar su calidad. De modelado, porque ajustar un modelo pequeño para una tarea con tanta variabilidad léxica y cultural implica decisiones de arquitectura, tamaño de modelo y técnica de *fine-tuning* (por ejemplo, LoRA) que afectan directamente el equilibrio entre eficiencia y calidad. Experimentales, porque comparar generadores de datos y técnicas de fusión de modelos exige un diseño controlado que aísle bien las variables relevantes. Y de arquitectura de sistema, porque el objetivo no es un script de entrenamiento local sino un servicio desplegable en la nube, con una API que pueda integrarse a otros sistemas (por ejemplo, un servicio de atención al cliente o una plataforma de salud) y que sea observable y escalable.

2.7. Preguntas de investigación

Este proyecto se enmarca principalmente como un problema científico y técnico, motivado por un problema social y organizacional real. Las preguntas de investigación que lo guían son:

- **PI1.** ¿Cómo afecta la elección del LLM utilizado para generar datos sintéticos de entrenamiento a la calidad de la traducción de un SLM ajustado para jerga y dialectos del español?
- **PI2.** ¿Puede la fusión de pesos (simple o guiada por destilación de conocimiento multi-maestro) consolidar en un único modelo el conocimiento de varios SLMs entrenados con datos sintéticos de distintas fuentes, igualando o superando el desempeño del mejor modelo individual?
- **PI3.** ¿Es posible obtener, mediante este enfoque, un modelo pequeño y eficiente cuya calidad de traducción de expresiones dialectales sea competitiva frente a sistemas de traducción de propósito general?

3. Estado del arte

3.1. Traducción automática y su fragilidad ante la jerga y los dialectos

Que a la traducción automática le cueste el texto que no es estándar no es nada nuevo: llevamos más de una década documentándolo. Michel y Neubig [7] armaron un banco de pruebas específico para medir cómo les iba a los sistemas de MT con texto ruidoso sacado de Reddit, y encontraron que basta con que la entrada no sea “limpia” para que hasta los sistemas más avanzados del momento terminaran produciendo traducciones desastrosas. Pero su banco de pruebas mete en la misma bolsa todo tipo de

ruido, sin distinguir de dónde viene. El trabajo más reciente sobre contenido generado por usuarios sí aísla específicamente el componente dialectal del error, y confirma el mismo patrón para español y árabe: buena parte de los errores de traducción “críticos” (esos que terminan invirtiendo el sentido del mensaje original) vienen justamente de expresiones dialectales [5,6]. Esta evidencia motiva directamente el problema que planteamos en la Sección 2, pero ojo —ninguno de estos trabajos propone una solución de *fine-tuning*, se quedan en caracterizar el error.

3.2. Traducción dialectal mediante ajuste fino de modelos de lenguaje

La propuesta que más se parece a la nuestra viene de trabajo reciente sobre dialectos de otros idiomas. Yakhni y Chehab [8] ajustaron el modelo Aya23-8B para traducir el dialecto libanés, probando instrucciones básicas, contrastivas y basadas en reglas gramaticales, y generaron parte de sus datos de entrenamiento con un LLM (Claude 3.5 Sonnet) a partir de un libro de gramática. Sus hallazgos nos sirven bastante: primero, que ajustar el modelo con datos culturalmente auténticos, aunque el conjunto sea chiquito, le gana consistentemente a entrenar con conjuntos más grandes pero traducidos de forma no nativa; y segundo, que entrenar por currículum (en fases sucesivas sobre distintos tipos de instrucción) no mejora nada, algo que ellos mismos atribuyen a olvido catastrófico y que sugieren podría resolverse entrenando modelos por separado y combinándolos después, justo la idea de fusión de modelos que nosotros queremos investigar de forma sistemática. Algo parecido pasa con el aromanian, donde Jerpelea et al. [9] muestran que un corpus paralelo diverso de unos 79 mil ejemplos ya alcanza para ajustar modelos preentrenados con buenos resultados, y con el šariš, donde Ondrejová y Šuppa [10] evalúan qué tan bien traducen los LLMs este dialecto eslovaco de bajo recurso. En el terreno de la jerga (que no es lo mismo que el dialecto regional, aunque están relacionados) Liang et al. [11] arman un banco de pruebas de traducción de jerga inglés-chino con más de 40 mil pares, donde su modelo especializado (SlangOWL) le gana a las líneas base sin ajuste fino.

Para el español, en cambio, lo que hay se queda en el reconocimiento dialectal, no en la traducción. Martínez et al. [12] publican un set de preguntas de opción múltiple para ver si los LLMs distinguen siete macro-variantes del español (andina, antillana, chilena, caribeña continental, mexicana/centroamericana, peninsular, rioplatense), pero sin ajustar ningún modelo para esa tarea. Zander-Marzano et al. [13], en cambio, sí lo hacen: ajustan cuatro LLMs pequeños (LLaMA, Gemma, Mistral, DeepSeek) para clasificar el registro formal/informal en cinco países hispano hablantes, y encuentran que la interferencia de jerga y morfología compartida entre países vecinos es una de las dos fuentes principales de error de clasificación, algo que refuerza justo lo que veníamos diciendo: la jerga y el dialecto complican hasta tareas más simples que la traducción. Pero ninguno de los dos entrena un modelo para traducir lo que detecta.

3.3. Generación de datos sintéticos y comparación de generadores

Como no existen corpus paralelos amplios para dialectos y jergas de bajo recurso, generar datos sintéticos con LLMs se ha vuelto la estrategia de cabecera, tanto para clasificación como para traducción de bajo recurso. Lo que no es tan común es tratar la elección del LLM generador como una variable experimental controlada, o sea, preguntarse en serio si importa cuál LLM usas para generar los datos. Poliakov y Shvai [16] compararon cuatro LLMs generadores (o4-mini, GPT-4.1, GPT-5, o3) para clasificar falacias lógicas, y encontraron diferencias de hasta 14 puntos porcentuales de F1 según cuál generador usaran, aunque hablamos de una tarea de clasificación cerrada, con solo cuatro generadores en juego. Allam et al. [17] usan solo dos generadores, pero se acercan más a lo que nosotros necesitamos porque es generación de texto libre: producen 80 mil pares de preguntas y respuestas médicas sintéticas con dos generadores distintos (GPT-4o y Gemini 2.5 Pro), y encuentran que los datos de GPT-4o dan consistentemente mejores puntajes F1 y menos alucinaciones. Estos dos trabajos son la evidencia más directa de que sí importa quién genera los datos sintéticos, y son los que nos motivan a plantear la Pregunta de Investigación 1; el problema es que ninguno mide el efecto sobre una tarea de traducción. En una línea relacionada, Schaffelder y Gatt [18] muestran que entrenar con datos sintéticos de fuentes diversas, en vez de una sola, ayuda a evitar el colapso de distribución y mantiene la diversidad del texto generado,

lo cual apoya indirectamente la idea de que combinar varias fuentes (ya sea mezclando el *dataset* o fusionando modelos, como planteamos aquí) puede ser mejor que depender de un solo generador.

3.4. Fusión de modelos (*model merging*)

Fusionar los pesos de modelos ajustados por separado se ha vuelto una alternativa liviana al entrenamiento conjunto. Las técnicas más básicas incluyen el promedio simple de pesos o *model soup* [19], la aritmética de tareas (*task arithmetic*), que representa cada ajuste fino como un vector de diferencia respecto al modelo base [20], y refinamientos que atenúan la interferencia entre modelos como TIES-Merging [21] y DARE [22]. La herramienta *mergekit* [23] junta estas técnicas y varias más en un formato accesible mediante archivos de configuración.

Aplicar todo esto a traducción automática, es poco común y la evidencia que hay es mixta. Tao et al. [25] muestran que fusionar modelos ajustados sobre distintos *datasets* funciona razonablemente bien como sustituto de entrenar sobre la mezcla completa de esos datos (con una correlación de Pearson de 0.78 en tareas de visión y 0.57 en tareas de lenguaje), pero sus experimentos son de clasificación y comprensión de lenguaje, no de traducción. En el terreno específicamente multilingüe, Gain et al. [24] fusionan modelos de traducción ajustados de forma independiente sobre distintos pares de idiomas, y encuentran una degradación de desempeño sustancial y desigual, sobre todo cuando el idioma de destino cambia entre los modelos fusionados, un resultado que nos sirve como advertencia, aunque su escenario (idiomas distintos) es diferente al nuestro (mismo par de idiomas, pero distinta fuente de datos sintéticos). A diferencia de esa fusión simple por promedio de pesos, el trabajo más reciente sobre fusión guiada por destilación de conocimiento multi-maestro [26] plantea la fusión como un problema de destilación, minimizando la diferencia entre lo que predice el modelo fusionado y lo que predice cada modelo especializado, pero solo lo evalúan en tareas de clasificación, y ellos mismos dejan la extensión a tareas generativas como trabajo futuro.

3.5. Síntesis y vacío identificado

La Tabla 1 resume el vacío que encontramos después de toda esta revisión: cada pieza de nuestra propuesta existe por separado en la literatura, pero la combinación específica que planteamos (traducir, no solo reconocer, jerga y dialecto del español, comparando de forma controlada los generadores sintéticos, y consolidando todo mediante fusión de modelos) no la encontramos en ninguna fuente.

Componente	Existe para...	Vacío identificado
Traducción de jerga/dialecto vía <i>fine-tuning</i>	Libanés [8], aromanian [9], šarīš [10], jerga en-zh [11]	No encontrado para español (solo hay reconocimiento [12, 13])
Comparación de LLMs como generadores sintéticos	Clasificación [16], diálogo médico [17]	No aplicado a traducción
Fusión de modelos entrenados con datos de distintas fuentes	Clasificación/QA [25]	No aplicado a traducción; fusión multilingüe estándar es frágil [24]

Tabla 1: Síntesis del vacío identificado en la literatura revisada.

4. Propuesta general de solución

4.1. Visión general

Lo que proponemos es construir y evaluar un *pipeline* completo de cuatro etapas. Primero, curar un banco de semillas con expresiones dialectales y de jerga junto con su traducción correcta. Segundo,

generar datos sintéticos de entrenamiento usando tres LLMs generadores distintos, cada uno trabajando por su cuenta sobre el mismo banco de semillas. Tercero, ajustar un SLM, en el rango de 1 a 3 mil millones de parámetros (estamos considerando Llama 3.2 3B, Qwen2.5 3B y Gemma 2 2B como candidatos), mediante LoRA, sobre cada uno de los tres conjuntos sintéticos que resulten. Y cuarto, consolidar los tres modelos especializados fusionando sus pesos, comparando una fusión simple (TIES/DARE vía `mergekit`) contra una fusión guiada por destilación multi-maestro. Todo esto lo pensamos como un servicio desplegable en la nube, expuesto mediante una API, no como un simple script de entrenamiento local, con observabilidad desde el diseño inicial (métricas de calidad por dialecto, latencia, tasa de retroalimentación).

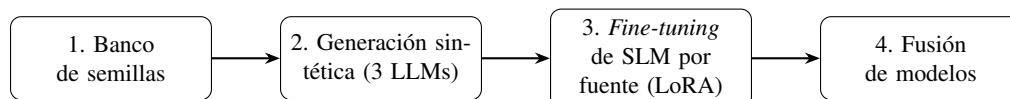


Figura 1: Pipeline conceptual de cuatro etapas de la propuesta: curación de semillas, generación sintética multi-LLM, ajuste fino por fuente, y fusión de modelos.

Como referencia para calibrar qué tan ambicioso puede ser el diseño experimental de las siguientes fases, declaramos el presupuesto de tiempo y cómputo que tenemos disponible en esta etapa como equipo: cada uno de los tres integrantes dedica entre 5 y 8 horas semanales al proyecto (entre 15 y 24 horas semanales combinadas), de forma compaginada con el resto de la carga académica del curso, sin dedicación exclusiva de nadie. En cuanto a cómputo, ninguna de las máquinas del equipo tiene GPU utilizable para ajustar un SLM de 3 mil millones de parámetros ni siquiera con LoRA (una tiene GPU integrada sin CUDA real, otra solo 2GB de VRAM), así que todo el entrenamiento, la fusión de modelos y la evaluación a gran escala se ejecutan sobre la capa gratuita de Google Colab (GPU T4/L4 según disponibilidad), sin costo de cómputo adicional para el proyecto.

4.2. Investigación y producto: dos preguntas distintas

Vale la pena separar dos preguntas que a primera vista parecen la misma, pero no lo son. La primera es por qué este proyecto vale la pena como trabajo de investigación de un semestre: porque nadie ha comparado de forma controlada distintos generadores de datos sintéticos ni ha probado la fusión de modelos para esta tarea específica, como se estableció en la Sección 3. La segunda es por qué alguien usaría lo que construyamos en vez de Google Translate, DeepL, o simplemente pedirle a ChatGPT que traduzca, y esa es una pregunta de producto, no de método, con una respuesta distinta.

La Tabla 2 resume esa segunda respuesta. Ninguna de esas diferencias depende de que nuestro método resulte ser el mejor posible; son ventajas que vienen directamente de haber elegido un SLM desplegable localmente en vez de envolver la API de un LLM grande.

Ninguna de las dos preguntas reemplaza a la otra. Si solo tuviéramos la respuesta de producto sin la de investigación, este sería un proyecto de conectar una interfaz a una API existente, justo lo que la rúbrica del curso marca como trivial. Y si solo tuviéramos la pregunta de investigación sin validar que el resultado sirve para algo real, nunca sabríamos si nuestros hallazgos importan fuera de un paper.

4.3. Decisiones técnicas y científicas a investigar

Hay varias decisiones que todavía no están resueltas y que van a ser parte del trabajo de investigación en sí: qué SLM base usar y con qué técnica de ajuste fino (LoRA vs. ajuste fino completo); cómo diseñar el proceso de “derivación” con el que cada generador expande las semillas en variantes de contexto, registro y tono; qué criterios usar para validar y filtrar los datos sintéticos (automáticos y con muestreo humano); qué técnica de fusión conviene más, sabiendo que fusionar adaptadores LoRA es más frágil que fusionar modelos completos; y cómo diseñar la arquitectura de despliegue (multi-tenant, observable, escalable) para que esto no se quede en un simple envoltorio sobre una API que ya existe. Un riesgo real en esta etapa es no tener suficiente acceso a GPU para hacer *fine-tuning* (LoRA) de varios SLMs en

Atributo	Google Translate / DeepL / ChatGPT	Nuestra propuesta
Conexión a internet	Obligatoria	No obligatoria; puede correr en el dispositivo
Datos sensibles (salud, legal)	Salen a la nube del proveedor	Pueden quedarse dentro de la infraestructura del cliente
Personalización	Caja negra, no ajustable	Se puede re-entrenar con la mezcla dialectal propia de cada cliente
Costo a gran volumen	Pago por llamada a un modelo grande	Modelo pequeño, barato de correr a escala
Auditabilidad	No se sabe con qué se entrenó	Se documenta el origen de cada dato de entrenamiento

Tabla 2: Diferencias de producto frente a las alternativas existentes, independientes del resultado del método de investigación.

paralelo; si eso pasa, vamos a mitigarlo priorizando un solo SLM base y reduciendo cuántos generadores sintéticos comparamos en la implementación inicial, o usando créditos de cómputo gratuitos o baratos (por ejemplo, Google Colab o créditos académicos).

4.4. Alternativa no basada en IA

Como punto de comparación, y para justificar por qué de verdad se necesita un componente inteligente, consideramos como alternativa un glosario o diccionario curado de equivalencias por dialecto, revisado por hablantes nativos. Es una opción más barata y totalmente explicable, pero no generaliza a construcciones nuevas ni a combinaciones de jerga y contexto que no se hayan previsto, que es justo la limitación que este proyecto quiere superar.

4.5. Atributos de calidad e hipótesis a evaluar

Más adelante en el proyecto vamos a evaluar, como mínimo: (i) la calidad de traducción, con métricas automáticas (BLEU, chrF) y evaluación humana de qué tanto se conservan los matices, comparando cada modelo individual, el modelo fusionado, y sistemas de referencia de propósito general; (ii) la eficiencia, comparando tamaño y latencia del SLM resultante frente a modelos más grandes; (iii) el efecto del generador sintético sobre la calidad del modelo (PI1); (iv) qué tan efectiva es la fusión frente al mejor modelo individual y frente a un modelo entrenado sobre la mezcla completa de los tres conjuntos de datos (PI2 y PI3); y (v) la portabilidad del modelo resultante, es decir, si puede correr en hardware modesto y sin conexión constante a internet, algo relevante en zonas rurales con conectividad limitada, como se discutió en la Sección 2.

La evaluación humana de retención de matices la vamos a hacer con un mínimo de tres hablantes nativos por variante dialectal cubierta en el banco de semillas, reclutados por plataformas de anotación remunerada (por ejemplo, Prolific) o por contactos directos del equipo, con un cuestionario de filtro que confirme su origen dialectal. Vamos a medir el acuerdo entre evaluadores con el coeficiente kappa de Cohen (si son dos anotadores) o de Fleiss (si son tres o más), y vamos a reportar ese nivel de acuerdo junto con los resultados de calidad.

5. Contribuciones

En términos generales, este proyecto espera dejar cuatro cosas concretas:

1. Evidencia de que sí es posible ajustar un modelo de lenguaje pequeño (SLM) para traducir jergas y dialectos del español, y no solo para reconocerlos o clasificarlos, que es hasta donde ha llegado la mayoría de trabajos que revisamos.
2. Una comparación controlada de cómo cambia la calidad de un modelo de traducción según qué LLM se use para generar sus datos de entrenamiento sintéticos. Este tipo de comparación ya se ha hecho para otras tareas, como clasificación de falacias lógicas o generación de diálogo médico [16, 17], pero nosotros la aplicamos por primera vez a la traducción de dialectos.
3. Evidencia sobre si es posible fusionar varios modelos de traducción (cada uno entrenado con datos sintéticos de una fuente distinta) en uno solo, ya sea con una fusión simple o guiada por destilación de conocimiento entre varios modelos maestros. Es un escenario que el trabajo previo en fusión de modelos generativos todavía no ha probado [26].
4. Un *pipeline* completo y documentado (generación de datos, validación y evaluación) que cualquier otro equipo pueda reutilizar si quiere extender este trabajo a otras variantes del español que nosotros no alcancemos a cubrir, o incluso, como base metodológica, a lenguas indígenas colombianas con escasez de datos todavía mayor (Sección 2) —algo fuera del alcance técnico de este proyecto, pero que el diseño del *pipeline* no descarta.
5. Más allá del aporte científico, un prototipo que, de resultar viable, ofrece ventajas de producto frente a Google Translate, DeepL o un LLM de propósito general: no depende de conexión a internet constante, no requiere que datos sensibles salgan a la nube de un tercero, es personalizable por cliente y es auditable en cuanto al origen de sus datos de entrenamiento (Sección 4, Tabla 2).

6. Trabajo futuro

Esta primera fase del proyecto se queda en la propuesta general, sin incluir arquitectura detallada, selección tecnológica definitiva, prototipo ni resultados experimentales.

En la siguiente fase vamos a desarrollar la arquitectura completa del sistema (negocio, datos, aplicaciones, tecnología, integración, seguridad y observabilidad) e implementar un prototipo: un servicio de traducción desplegado en la nube, expuesto mediante API, capaz de atender varias solicitudes al mismo tiempo y con instrumentación para observar la calidad de traducción por dialecto. Queremos evitar a propósito que el prototipo se quede en un simple script de entrenamiento corriendo en una máquina; la idea es que el sistema demuestre decisiones de arquitectura de nivel empresarial (escalabilidad, seguridad, costos y despliegue reproducible).

En esa fase, el incremento mínimo viable va a cubrir un solo generador sintético de los tres LLMs que estamos considerando y una sola técnica de fusión (fusión simple vía `mergekit`).

La comparación completa entre los tres generadores y entre las dos técnicas de fusión (simple y guiada por destilación multi-maestro) queda para la fase final del proyecto, sujeta a cuánto tiempo y cómputo tengamos disponible como equipo para ese momento. En esa última fase presentaremos los resultados cuantitativos (métricas automáticas y comparación entre generadores, modelos individuales y modelo fusionado) y cualitativos (evaluación humana de qué tanto se conservan los matices dialectales), obtenidos con el diseño experimental descrito en la Sección 4, junto con el análisis de costos, riesgos, gobernanza y las limitaciones del enfoque.

7. Conclusiones

Este documento presentó la propuesta inicial de un proyecto que busca traducir jerga y dialectos del español y el inglés mediante un modelo de lenguaje pequeño ajustado sobre datos sintéticos generados por varios LLMs, y explorar si esa información se puede consolidar fusionando modelos. La revisión de literatura que hicimos (fuentes científicas en inglés, literatura técnica y reportes de industria) nos

permitió sustentar el problema con evidencia cuantitativa y cualitativa, y también identificar un vacío específico en la intersección de tres líneas de trabajo que ya existen por separado: traducción dialectal mediante *fine-tuning*, comparación de generadores de datos sintéticos, y fusión de modelos entrenados con datos heterogéneos. Hasta donde pudimos establecer, ninguna de las tres se ha aplicado todavía a la traducción de jerga y dialecto del español. La propuesta implica decisiones de datos, modelado, diseño experimental y arquitectura de sistema que iremos desarrollando en las siguientes fases, empezando por una arquitectura de nivel empresarial y un prototipo funcional en la etapa siguiente.

Referencias

- [1] Price-Wise, G. (2008). *Language, Culture, And Medical Tragedy: The Case Of Willie Ramirez*. Health Affairs Forefront. <https://www.healthaffairs.org/doi/10.1377/forefront.20081119.000463/>
- [2] AMS Resources (2026). *Spanish dialects in U.S. legal and medical interpreting: when matching matters*. <https://accessmultilingual.com/resources/spanish-dialects-in-us-legal-and-medical-interpreting>
- [3] U.S. Census Bureau (2022). *Languages We Speak in the United States*. <https://www.census.gov/library/stories/2022/12/languages-we-speak-in-united-states.html>
- [4] Migration Policy Institute (2023). *Language Diversity and English Proficiency in the United States*. <https://www.migrationpolicy.org/article/language-diversity-and-english-proficiency-united-states>
- [5] Saadany, H., Tantawy, A., Orăsan, C. (2024). *Cyber Risks of Machine Translation Critical Errors: Arabic Mental Health Tweets as a Case Study*. arXiv:2405.11668.
- [6] Saadany, H., Orăsan, C., Caro Quintana, R., do Carmo, F., Zilio, L. (2021). *Challenges in Translation of Emotions in Multilingual User-Generated Content: Twitter as a Case Study*. arXiv:2106.10719.
- [7] Michel, P., Neubig, G. (2018). *MTNT: A Testbed for Machine Translation of Noisy Text*. arXiv:1809.00388.
- [8] Yakhni, S., Chehab, A. (2025). *Fine-Tuning LLMs for Low-Resource Dialect Translation: The Case of Lebanese*. arXiv:2505.00114.
- [9] Jerpelea, A., Radoi, A., Nisioi, S. (2025). *Dialectal and Low-Resource Machine Translation for Romanian*. COLING 2025 / arXiv:2410.17728.
- [10] Ondrejová, V., Šuppa, M. (2024). *Can LLMs Handle Low-Resource Dialects? A Case Study on Translation and Common Sense Reasoning in Šariš*. Proceedings of VarDial 2024, ACL Anthology 2024.vardial-1.11.
- [11] Liang, J., Meng, F., Wang, Y., Zhou, J. (2025). *SlangDIT: Benchmarking LLMs in Interpretative Slang Translation*. arXiv:2505.14181.
- [12] Martínez, A., Mayor-Rocher, S., Pozo Huertas, C., Melero, M., Grandury, M., Reviriego, P. (2025). *Spanish is not just one: A dataset of Spanish dialect recognition for LLMs*. Data in Brief, 63, 112088.
- [13] Zander-Marzano, D., Hernández-Orallo, J., Martínez-Plumed, F., et al. (2026). *From formal to informal: how register shift confuses LLMs in Spanish dialect recognition*. International Journal of Data Science and Analytics, 22, 205.

- [14] Kawasaki, Y. (2026). *Digital Linguistic Bias in Spanish: Evidence from Lexical Variation in LLMs*. arXiv:2602.09346.
- [15] Rest of World (2025). *Latin America is building LatamGPT to rival ChatGPT*. <https://restofworld.org/2025/chatgpt-latin-america-alternative-latamgpt/>
- [16] Poliakov, M., Shvai, N. (2025). *MisSynth: Improving MISSCI Logical Fallacies Classification with Synthetic Data*. arXiv:2510.26345.
- [17] Allam, A., et al. (2025). *Scaling Arabic Medical Chatbots Using Synthetic Data: Enhancing Generative AI with Synthetic Patient Records*. arXiv:2509.10108.
- [18] Schaffelder, M., Gatt, A. (2026). *Synthetic Eggs in Many Baskets: The Impact of Synthetic Data Diversity on LLM Fine-Tuning*. Findings of the Association for Computational Linguistics: ACL 2026, pp. 7265–7293. arXiv:2511.01490.
- [19] Wortsman, M., et al. (2022). *Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time*. Proceedings of the 39th International Conference on Machine Learning (ICML), pp. 23965–23998.
- [20] Ilharco, G., Ribeiro, M.T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., Farhadi, A. (2022). *Editing Models with Task Arithmetic*. arXiv:2212.04089.
- [21] Yadav, P., Tam, D., Choshen, L., Raffel, C., Bansal, M. (2023). *TIES-Merging: Resolving Interference When Merging Models*. Advances in Neural Information Processing Systems (NeurIPS) 36.
- [22] Yu, L., Yu, B., Yu, H., Huang, F., Li, Y. (2024). *Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch*. Proceedings of the 41st International Conference on Machine Learning (ICML).
- [23] Goddard, C., Siriwardhana, S., Ehghaghi, M., Meyers, L., Karpukhin, V., Benedict, B., McQuade, M., Solawetz, J. (2024). *Arcee’s MergeKit: A Toolkit for Merging Large Language Models*. Proceedings of EMNLP 2024, Industry Track, pp. 477–485.
- [24] Gain, B., Ekbal, A., Singh, T.N. (2026). *One Model to Translate Them All? A Journey to Mount Doom for Multilingual Model Merging*. arXiv:2604.02881.
- [25] Tao, Z.S., Vinken, K., Yeh, H.W., Cooper, A., Boix, X. (2025). *Merge to Mix: Mixing Datasets via Model Merging*. arXiv:2505.16066.
- [26] Dalili, S.A., Mahdavi, M. (2025). *Model Merging via Multi-Teacher Knowledge Distillation*. arXiv:2512.21288.
- [27] Google (2025). *Bringing state-of-the-art Gemini translation capabilities to Google Translate*. Google Blog. <https://blog.google/products/search/gemini-capabilities-translation-upgrades/>
- [28] DeepL Help Center (2026). *DeepL Translator languages*. <https://support.deepl.com/hc/en-us/articles/360019925219-DeepL-Translator-languages>
- [29] Jimenez, I.D., Kern, C. (2026). *Dialect and Gender Bias in YouTube’s Spanish Captioning System*. arXiv:2602.24002.
- [30] Linglass (2026). *YouTube Auto-Translate Subtitles Not Working? Here’s Why*. <https://linglass.app/blog/youtube-auto-translate-not-working>

- [31] El Tiempo (2025). *Lenguas indígenas, un tesoro nacional en peligro*. <https://www.eltiempo.com/colombia/otras-ciudades/lenguas-indigenas-un-tesoro-nacional-en-peligro-3417164>
- [32] Cambio Colombia (2026). *Gobierno Petro conectó 3,5 millones de hogares y amplía el acceso a internet en zonas rurales*, con base en la Encuesta de Calidad de Vida 2025 del DANE. <https://cambiocolombia.com/contenido-especial/articulo/2026/5/gobierno-petro-conecto-35-millones-de-hogares-y-amplia-el-acceso-a-internet-en-zonas-rurales/>